Chapter 7

Coursework 1 Handout

Distributed flocking control: building a simulated drone swarm with FreeRTOS tasks, LoRa communication and MQTT visualisation

Rationale In this coursework, we will work together on building a low-level, real-time networked system on the ESP32 using ESP-IDF and FreeRTOS. Instead of mundane GPIO exercises, we will create a small onboard simulation of multiple networked drones performing flocking behaviours and making local decisions. You will design concurrent tasks with deadlines, exchange neighbour state over a low-bandwidth radio link, and publish state to an MQTT channel so we can visualise the results.

The module booklet specifies that this exercise will develop practical skills applied to produce low-level code on wireless microcontrollers, measuring the performance of this in the context of drone simulation. This handout concretises those requirements on the ESP32 platform: you will implement concurrent real-time tasks, inter-task communication, radio messaging, and state publication, and you will evaluate latency, jitter, availability, stability, energy use and resilience to adversarial attacks.

Learning objectives By the end of this coursework, you will be able to:

- Design a real-time task structure for a simulated drone on the ESP32.
- Implement inter-task communication and scheduling with FreeRTOS.
- Simulate drone physics in 3D (translational motion with velocity components and yaw).
- Implement a flocking controller based on local neighbour state.
- Use Wi-Fi/LoRa and MQTT for communication and telemetry.
- Develop a threat model and test security considerations in peer adversarial challenges.

Assumptions on drone model For this coursework, the drones will be modelled as simplified multirotors (quadcopter-style rotorcraft). The physics integration task updates position in x, y, z from commanded velocities and keeps track of heading because, whilst flight dynamics are generally isotropic, cameras and sensing need not be. Hover and full 3D translation are

permitted, but we simplify orientation to a single yaw heading. Pitch and roll dynamics need not be modelled (though you can if you wish), but no fixed-wing aerodynamics (lift, stall, coordinated turns) should be included. Environmental effects, such as wind, can also be elided. This keeps the focus on real-time scheduling, networking, and security while remaining realistic for swarm robotics behaviours.

High level system architecture

- You will use ESP32 devices to simulate drones and exchange neighbour state over LoRa.
- You will send MQTT telemetry over Wi-Fi (or via a gateway that relays LoRa to MQTT) to a broker I will provide.
- I will provide a simple flock visualiser based on information obtained from the MQTT broker.

Functional requirements

1. **Multiple FreeRTOS tasks (minimum 4),** with distinct roles and priorities:
   - **Physics integration task** (50 Hz) - integrates 3D kinematics for position x, y, z, velocity components $\mathbf{v} = (v_x, v_y, v_z)$, and yaw heading ψ; applies velocity and yaw rate commands.
   - **Flocking control task** (10 Hz) - computes 3D velocity and yaw rate commands from local neighbour state using a standard flocking method (e.g. Reynolds cohesion-alignment-separation, with optional extensions such as obstacle avoidance).
   - **Radio I/O task** (2–5 Hz) - packages and broadcasts local state over LoRa and ingests neighbour state; maintains a short-lived neighbour table with timestamps. The neighbour table should contain all neighbours currently active with our team id.
   - **Telemetry task** (2–5 Hz) - publishes current state to MQTT for visualisation directly over Wi-Fi.
2. **Inter-task communication** - use queues, stream buffers, or mailboxes to pass state between tasks; avoid unbounded blocking in high-priority tasks.
3. **Wireless neighbour exchange (LoRa)** - periodically broadcast compact state packets and process received packets from neighbours. Maintain sequence numbers and timeouts to estimate availability.
4. **MQTT state output** - publish position and velocity to a distinct topic namespace for display in a dashboard.

The values in brackets contain the expected task frequencies. Your implementation should remain close to these targets and provide evidence that they are met.

LoRa packet format and numeric encodings

To give us a common starting point for interaction, here is a suggested packet format for your LoRa packets. You are free to change it if you wish, but because this forms the shared basis for communication and others may have implemented it as written, you *must* get the positive agreement of everyone in the class before doing so.

My suggested fields, sizes and encodings are:

```
[ version:1 ]         % uint8  - protocol version
[ flags:1 ]           % uint8  - bitflags / status
[ team_id:1 ]         % uint8  - zero is everyone; you organise anything else
[ node_id:6 ]         % uint8  - use the 48-bit MAC address of your node
[ seq:2 ]             % uint16 - sequence number
[ ts_s:4 ]            % uint32 - Unix seconds (NTP-synced)
[ ts_ms:2 ]           % uint16 - milliseconds remainder (0..999)
[ x_mm:4 ]            % int32  - x position in millimetres
[ y_mm:4 ]            % int32  - y position in millimetres
[ z_mm:4 ]            % int32  - z position in millimetres
[ vx_mms:4 ]          % int32  - x velocity in millimetres per second
[ vy_mms:4 ]          % int32  - y velocity in millimetres per second
[ vz_mms:4 ]          % int32  - z velocity in millimetres per second
[ heading_cd:2 ]      % uint16 - heading in centi-degrees (0..35999)
[ mac_tag:4 ]         % 4 byte truncated MAC (see security)
```

Notes:

- Integer encodings use big-endian (network byte order).
- `team_id = 0` is reserved – it is what we will use to ensure that all ESP32s in range are included in the drone simulation. You are free to set different values for this when testing, but be aware that others might have chosen the same number.
- Positions and velocities are *positive*, unsigned, integer values (mm, mm/s) to avoid floating point ambiguity and define the origin within the space.
- To make the visualisation easier, please restrict movement to within a 100m × 100m × 100m box.
- Your values for velocities (and accelerations if you use them) must be realistic for a multi-rotor drone. You should justify your choices
- The timestamp is Unix time synchronised by NTP; include both seconds and millisecond remainder for millisecond resolution.
- The `version` and `flags` bytes enable forward compatibility and small status bits.
- The `heading_cd` field represents yaw heading only (rotation about the vertical axis). Pitch and roll are not modelled in this simulation.
- The `mac_tag` is a truncated authentication tag computed as specified below.
- LoRa neighbour broadcasts should occur at 2–5 Hz. Choose a rate within this range and justify your choice in the report. You should compute the expected airtime for your LoRa parameters (SF, BW, CR) and keep average on-air time per minute at reasonable levels; provide these calculations in the report.

MQTT topics and payload

Let us turn to the details of MQTT publishing so we can visualise the data. These must be common to everyone and so please use the following:

MQTT broker: MQTT telemetry is mandatory and must use my broker. I will publish details of this on Moodle.

Topic: Please use `COMP0221/flock/0/state` as the topic to which you publish

Payload: Please use a JSON packet with the fields as defined for LoRa above. Use field names exactly as written above.

Duty-cycle: Broadcast telemetry at 2 Hz by default. Allow for it to increase up to 5 Hz if this turns out to be needed.

Security primitive and key management

To make authentication realistic but manageable, we will use ESP-IDF primitives:

- **Primitive:** AES-128-CMAC over the packet bytes prior to the MAC field.
- **MAC:** truncate the 128-bit CMAC to the lower 32 bits (4 bytes), appended as `mac_tag`.
- **Key provisioning:** each team should use a per-team pre-shared 16-byte symmetric key of their own choosing. I may ask for this, but do not disclose keys in code repositories or blog posts.
- **Verification:** receivers should recompute AES-CMAC and compare to the truncated 4-byte tag.

In your report, you must discuss trade-offs between tag size, security and airtime in this choice of security.

Adversarial testing: activities and ethics

We will include adversarial testing as part of this coursework, but it must follow ethical rules (see the final section of the booklet). Before you run any adversarial tests, you will need to submit an ethics plan on Moodle and obtain demonstrator sign-off.

Possible adversarial attacks: You should feel free to attack in any way you please, subject to the short ethics plan. However, here are some suggestions to get you started.

1. **Replay attack:** capture and replay previously observed packets.
2. **Packet spoofing:** craft forged packets to inject false neighbour state; compare behaviour with and without correct MAC.
3. **Replay with modified timestamp:** create ordering confusion and observe flock instability.
4. **Flooding / resource exhaustion:** controlled high-rate packet send to observe missed deadlines (must respect duty-cycle limits).
5. **Protocol confusion:** send packets with unknown version or flags and observe fallback behaviour.

Note: **MQTT tampering** is off-limits, since it's our only way of seeing the effect of an attack on motion. Please *do not* attempt to publish false telemetry to the visualisation channel.

Mandatory ethics plan and logging:

- Submit a short ethics plan on Moodle before Week 6 describing planned tests, expected impact and recovery steps. You need not share this with those whom you attack.
- Explicitly agree with others a schedule for attacking their systems. You should not do it without their knowledge - though the details of how they are being attacked need not be shared.
- During any attack run, the attacker should capture logs including: attack details, timestamps, captured packets, effects on flocking as measured by capturing MQTT packets. The defender should create a short incident report explaining effects as measured locally and reflection on that.
- Devices must be restorable within 15 minutes max (and preferably within a minute); we will refuse unsafe tests.

Flocking behaviour

The flock consists of all nodes currently accessible over the LoRa channel - this number will vary depending on the environment and the team id you set, which you can configure it as needed. We will implement Reynolds-style[1,2,3] cohesion, alignment, and separation. Try tuning the weights to achieve stable flocking without collisions - we might want to agree these together. As an aside, explore the effects of these parameters at https://eater.net/boids or https://github.com/rystrauss/boids.

Flocking behaviour depends on neighbour state, so make sure your neighbour exchange is working correctly before implementing the flocking controller.

Performance evaluation

You must collect and report evidence of:

- Latency and jitter of periodic tasks.
- Availability of neighbour messages.
- Stability of flocking (distance to centroid, heading alignment, minimum separation).
- Energy consumption over representative runs.

Include measurements for both normal operation and during selected adversarial tests, so we can see how the system behaves under stress

Assessment criteria (expanded)

The marking rubric in the module booklet (50 marks) applies. To achieve highly:

- **Correctness (10 marks)** – your simulation must run reliably with stable flocking.

[1] Reynolds, C. W. (1987). Flocks, Herds and Schools: A Distributed Behavioral Model. In Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '87), pp. 25-34.
[2] https://en.wikipedia.org/wiki/Boids
[3] http://www.kfish.org/boids/pseudocode.html

- **Timing (5 marks)** – use FreeRTOS periodic scheduling and provide clear evidence of deadlines met.
- **Communication (5 marks)** – neighbour packets and MQTT telemetry must be correct and measured.
- **Security (5 marks)** – define a threat model, conduct ethical adversarial testing, and reflect on results.
- **Technical report (10 marks)** – maximum 8 pages, clear structure, diagrams, measurements, critical reflection.
- **Blog post (5 marks)** – 300–500 words, peer-visible on Moodle, insightful and reflective.
- **Progress marks (10 marks)** – weekly binary assessment in labs, demonstrating functionality and explaining your design.

Deliverables

At the end of the coursework, you will submit:

- A private GitHub repository containing your full codebase.
- A technical report (PDF, maximum 8 pages).
- A blog post (300–500 words) submitted via Moodle.

Your technical report should include results and discussion of your adversarial tests and their impact on system behaviour

Submission and timeline

Submission Timeline:

Week 1: Coursework Issued

Week 6: Submit your ethics plan before beginning adversarial testing.

Week 10: Coursework Due (via Moodle)

Weekly marking:

Week 2: Check basic examples running.

Week 3: Demonstrate multiple FreeRTOS tasks with correct timing in lab.

Week 5: Show neighbour exchange functioning in lab.

Week 7: Demonstrate flocking behaviour in lab.

Week 9: Demonstrate adversarial activity in lab

Week 10: Submit your final deliverables (GitHub, report, blog post) via Moodle.

Queries

This is the first time we have run this coursework so, whilst I have tried to cover as much as possible above, there may be details that still need to be addressed. Please ask questions on Moodle throughout and we'll resolve them there.

CW1 Description from the module booklet

Title: Building and Securing a Networked Robotic System on the ESP32
Weight: 50%
Format: Individual

Overview:

You will design, implement and document key elements of a cyber-physical robotic system on the ESP32 platform. This system must include at least one sensor input, a networked communication protocol (e.g. Wi-Fi or ESP-NOW), and one or more actuators. The system must meet real-time constraints for task execution and communication.

In addition to the technical implementation, you will:

- Design and apply a basic threat model to your system.
- Participate in a peer adversarial challenge (under ethical constraints).
- Publish a blog post summarising a key aspect of your system, design process or security testing.

Submission Components:

1. Full working codebase (GitHub)
2. Technical report (max 8 pages)
3. Blog post (300–500 words), peer-visible and submitted via Moodle

Your progress will also be assessed on a regular basis in the lab session - you will be binary marked on progress and the ability to explain what you have done.

Blog Post Prompts (choose one or pick an appropriate one of your own):

- "The biggest design challenge in my system was..."
- "How I approached secure messaging with the ESP32"
- "Debugging timing issues in FreeRTOS"
- "What I learned from being 'attacked' by a peer"
- "Trade-offs: Real-time constraints vs security overhead"

Marking Rubric (50 marks):

Criteria	Marks
Correctness and functionality: all required system components implemented and operating reliably	10
Timing guarantees: latency, jitter and task deadlines are analysed and met	5
Communication protocol: appropriate use of wireless protocols and performance measured	5
Security: appropriate threat model, mitigations and evidence of adversarial robustness	5

Technical report: clarity, justification, evaluation and reflection	10
Blog post: communicates insight clearly, reflects on design/learning, accessible to a broader audience	5
Progress marks	10
Total	**50**

Blog Guidance:

- Write for your peers and future students: clarity and honesty are key.
- You may use diagrams, photos, and external links.
- Cite tools or references where appropriate.
- Posts will be reviewed for clarity, accuracy, and reflective value.

Moderation and Publication:

Blog posts will be hosted internally on Moodle. A selection of the most insightful posts may be published (with student consent) on an externally-facing website. Posts may not include technical details that would compromise system security or violate ethical guidelines.

Word count: 300–500 words. Format is free (Markdown recommended).

Ethical Constraints on Adversarial activities: All adversarial testing must comply with the following ethical constraints:

- Attacks must not brick devices or corrupt storage.
- All attacks must be declared in advance in a signed ethics plan.
- Attacks must be repeatable and logged.
- Devices must be restorable within 15 minutes post-test.